

stdio vs. Streamable HTTP

The two transport mechanisms used by the Model Context Protocol (MCP) — and by client/server tool integrations generally

Aspect	stdio	Streamable HTTP
What it is	Communication over the process's standard input/output streams	An HTTP-based transport where a single endpoint handles requests and can stream responses (via chunked responses or SSE)
Where it runs	Locally: client launches the server as a subprocess on the same machine	Over a network: client and server can be on different machines, connected via a URL
Connection model	Client spawns the process and talks to it directly through pipes for the process lifetime	Client sends HTTP POST requests to a server endpoint; server can stream events back (e.g. Server-Sent Events)
Setup	Simple — just a command to run; no ports, hosting, or network config needed	Requires a hosted, reachable server (URL, port, possibly TLS) that stays running
Multiple clients	One client per process instance — not shared across users	One server can serve many clients/sessions concurrently
Authentication	None needed — trust is implicit since it's a local subprocess	Supports standard HTTP auth: bearer tokens, OAuth, API keys, headers, etc.
Latency & overhead	Very low — direct OS pipes, no network stack	Higher — HTTP handshake, headers, and (if used) TLS negotiation
Lifecycle	Tied to the client: server process starts and stops with the client session	Independent: server runs continuously and can outlive any single client session
Typical use case	Local tools, desktop apps, CLI-based MCP servers (e.g. filesystem, local dev tools)	Remote/cloud MCP servers, third-party integrations, services shared across users or devices
Debugging	Inspect raw stdin/stdout stream directly; simple but few dedicated tools	Standard HTTP tooling applies: browser dev tools, curl, proxies, request logging

In short

stdio is the simplest transport: the client runs the server as a local subprocess and talks to it through standard input/output. No network, no auth, minimal overhead — ideal when client and server live on the same machine. **Streamable HTTP** is built for remote, networked scenarios: the server is a long-running HTTP endpoint that can stream data back to potentially many clients, with standard web authentication and tooling available, at the cost of more setup and per-request overhead. As a rule of thumb: use stdio for local, single-user tools; use Streamable HTTP when the server needs to be reachable remotely or shared across multiple clients.